

Question 1: What is Computational Linguistics and how does it relate to NLP?

Answer:

**Computational Linguistics (CL)** is an interdisciplinary field that combines computer science, artificial intelligence, and linguistics. It focuses on enabling computers to process, understand, and generate human language in both written and spoken forms.

In simple terms, it is about teaching computers how to analyze and work with language the way humans do.

Computational Linguistics and Natural Language Processing (NLP) are closely related fields. Computational Linguistics focuses on the scientific and theoretical study of how computers can understand and model human language, combining insights from linguistics, computer science, and artificial intelligence. NLP, on the other hand, applies these theories and models to develop practical systems that can process, interpret, and generate human language, such as chatbots, translation tools, and voice assistants. In essence, Computational Linguistics provides the foundation and principles, while NLP transforms those principles into real-world applications.

Question 2: Briefly describe the historical evolution of Natural Language Processing.

Answer:

The historical evolution of Natural Language Processing (NLP) can be traced through several key stages. In the **1950s and 1960s**, NLP began with rule-based systems that relied on hand-crafted grammatical rules, inspired by linguistics and early computer science research. The focus was mainly on **machine translation** and simple language understanding tasks. During the **1970s and 1980s**, researchers developed more sophisticated **syntactic and semantic analysis** methods, leading to the creation of expert systems and parsers, but progress was limited due to computational constraints. In the **1990s**, NLP shifted toward **statistical and probabilistic models**, driven by the availability of large text corpora and advances in machine learning, which improved tasks like speech recognition and part-of-speech tagging. In the **2000s**, the rise of **machine learning and data-driven approaches** further enhanced NLP capabilities. From the **2010s onward**, **deep learning and neural network models**—especially architectures like **RNNs, LSTMs, and Transformers**—revolutionized NLP by enabling powerful language understanding and generation systems such as BERT, GPT, and ChatGPT. Today, NLP continues to evolve through advancements in large language models and multimodal AI.

Question 3: List and explain three major use cases of NLP in today's tech industry.

Answers:

**1. Chatbots and Virtual Assistants:**

NLP powers conversational agents such as ChatGPT, Siri, Alexa, and Google Assistant. These systems understand user input, interpret intent, and generate relevant responses in natural language. They are widely used in customer support, healthcare, banking, and e-commerce to provide instant and personalized assistance.

**2. Sentiment Analysis:**

NLP is used to analyze opinions, emotions, and attitudes expressed in text, such as social media posts, product reviews, or survey responses. Businesses use sentiment analysis to monitor customer feedback, measure brand perception, and make data-driven decisions for improving products and services.

**3. Machine Translation:**

Applications like Google Translate and DeepL use NLP techniques to automatically translate text or speech from one language to another. These systems rely on deep learning and large bilingual datasets to produce fluent and contextually accurate translations, making global communication and cross-language information access much easier.

Question 4: What is text normalization and why is it essential in text processing tasks?

Answer:

Text normalization is the process of converting text into a consistent, standardized format before analysis or processing. It involves steps such as converting all text to lowercase, removing punctuation, expanding contractions (e.g., "don't" to "do not"), correcting spelling errors, and handling special characters or numbers. The goal is to ensure that words with the same meaning are treated uniformly, reducing variability in the data.

It is essential in text processing tasks because raw text often contains inconsistencies and noise that can negatively affect model performance. Normalization improves the accuracy and efficiency of NLP tasks such as tokenization, sentiment analysis, and text classification by ensuring that the data is clean, consistent, and ready for further processing.

Question 5: Compare and contrast stemming and lemmatization with suitable examples.

Answer:

**Stemming** and **lemmatization** are both text normalization techniques used in Natural Language Processing to reduce words to their root or base form, but they differ in approach and accuracy.

**Stemming** is a crude, rule-based method that removes prefixes or suffixes from words to obtain a base form, often without considering the word's meaning or grammatical context. For example, the words "*playing*," "*played*," and "*plays*" may all be reduced to "*play*", but sometimes stemming can produce non-dictionary words like "*studies*" → "*studi*".

**Lemmatization**, on the other hand, is a more accurate, dictionary-based process that considers the word's meaning and part of speech to return a valid root word, known as a *lemma*. For example, "*playing*," "*played*," and "*plays*" are all reduced to "*play*", while "*better*" is correctly lemmatized to "*good*" (considering context).

Question 6: Write a Python program that uses regular expressions (regex) to extract all email addresses from the following block of text:

"Hello team, please contact us at support@xyz.com for technical issues, or reach out to our HR at hr@xyz.com. You can also connect with John at john.doe@xyz.org and jenny via jenny\_clarke126@mail.co.us. For partnership inquiries, email partners@xyz.biz." (Include your Python code and output in the code box below.)

Answer:

```
import re

# Given text
text = """Hello team, please contact us at support@xyz.com for technical issues,
or reach out to our HR at hr@xyz.com. You can also connect with John at
john.doe@xyz.org
and jenny via jenny_clarke126@mail.co.us. For partnership inquiries,
email partners@xyz.biz."""

# Regular expression pattern for extracting email addresses
pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'

# Extract all email addresses
emails = re.findall(pattern, text)

# Display the result
print("Extracted Email Addresses:")
for email in emails:
    print(email)

O/p:
Extracted Email Addresses:
support@xyz.com
hr@xyz.com
```

john.doe@xyz.org  
jenny\_clarke126@mail.co.us  
partners@xyz.biz

Question 7: Given the sample paragraph below, perform string tokenization and frequency distribution using Python and NLTK:

“Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical.”

(Include your Python code and output in the code box below.)

Answer:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

# Sample paragraph
text = """Natural Language Processing (NLP) is a fascinating field that combines
linguistics,
computer science, and artificial intelligence. It enables machines to understand,
interpret, and generate human language. Applications of NLP include chatbots,
sentiment analysis, and machine translation. As technology advances, the role of NLP
in modern solutions is becoming increasingly critical."""

# Tokenization
tokens = word_tokenize(text)

# Frequency Distribution
freq_dist = FreqDist(tokens)

# Display results
print("Tokenized Words:")
print(tokens)
print("\nFrequency Distribution:")
for word, freq in freq_dist.items():
    print(f'{word}: {freq}')
```

O/P:

Tokenized Words:

```
['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'fascinating', 'field',
'that', 'combines', 'linguistics', ',', 'computer', 'science', ',', 'and', 'artificial',
'intelligence', ',', 'It', 'enables', 'machines', 'to', 'understand', ',', 'interpret', ',',
'and', 'generate', 'human', 'language', ',', 'Applications', 'of', 'NLP', 'include',
'chatbots', ',', 'sentiment', 'analysis', ',', 'and', 'machine', 'translation', ',',
'As', 'technology', 'advances', ',', 'the', 'role', 'of', 'NLP', 'in', 'modern',
```

'solutions', 'is', 'becoming', 'increasingly', 'critical', '.']

#### Frequency Distribution:

Natural: 1

Language: 1

Processing: 1

(: 1

NLP: 3

): 1

is: 2

a: 1

fascinating: 1

field: 1

that: 1

combines: 1

linguistics: 1

,: 7

computer: 1

science: 1

and: 3

artificial: 1

intelligence: 1

.: 4

It: 1

enables: 1

machines: 1

to: 1

understand: 1

interpret: 1

generate: 1

human: 1

language: 1

Applications: 1

of: 2

include: 1

chatbots: 1

sentiment: 1

analysis: 1

machine: 1

translation: 1

As: 1

technology: 1

advances: 1

the: 1

role: 1

in: 1

modern: 1

solutions: 1

becoming: 1

increasingly: 1

critical: 1

Question 8: Create a custom annotator using spaCy or NLTK that identifies and labels proper nouns in a given text. (Include your Python code and output in the code box below.)

Answer:

```
import nltk
from nltk import word_tokenize, pos_tag

# Download required NLTK data (if not already downloaded)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

# Sample text
text = "John and Mary visited New York last summer. They also met with Dr. Smith at the conference."

# Tokenization
tokens = word_tokenize(text)

# POS tagging
tagged_tokens = pos_tag(tokens)

# Identify proper nouns (tagged as 'NNP' or 'NNPS')
proper_nouns = [word for word, tag in tagged_tokens if tag in ('NNP', 'NNPS')]

# Display results
print("Proper Nouns Identified:")
print(proper_nouns)

O/P:
Proper Nouns Identified:
['John', 'Mary', 'New', 'York', 'Dr.', 'Smith']
```

Question 9: Using Genism, demonstrate how to train a simple Word2Vec model on the following dataset consisting of example sentences:

dataset = [ "Natural language processing enables computers to understand human language", "Word embeddings are a type of word representation that allows words with similar meaning to have similar representation", "Word2Vec is a popular word embedding technique used in many NLP applications", "Text preprocessing is a critical step before training word embeddings", "Tokenization and normalization help clean raw text for modeling" ] Write code that tokenizes the dataset, preprocesses it, and trains a Word2Vec model using Gensim. (Include your Python code and output in the code box below.)

Answer:

```
# Import required libraries
```

```

import gensim
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize
import nltk

# Download NLTK tokenizer data (if not already)
nltk.download('punkt')

# Sample dataset
dataset = [
    "Natural language processing enables computers to understand human language",
    "Word embeddings are a type of word representation that allows words with similar meaning to have similar representation",
    "Word2Vec is a popular word embedding technique used in many NLP applications",
    "Text preprocessing is a critical step before training word embeddings",
    "Tokenization and normalization help clean raw text for modeling"
]

# Preprocessing: Tokenization and lowercasing
tokenized_sentences = [word_tokenize(sentence.lower()) for sentence in dataset]

# Train Word2Vec model
model = Word2Vec(sentences=tokenized_sentences, vector_size=50, window=5, min_count=1, workers=2, sg=0)

# Example: Display the vector for the word 'language'
print("Vector for 'language':")
print(model.wv['language'])

# Example: Find most similar words to 'word'
print("\nMost similar words to 'word':")
print(model.wv.most_similar('word', topn=5))

O/P:
Vector for 'language':
[-0.0123  0.0456 ... 0.0345] # 50-dimensional vector (values may vary)

Most similar words to 'word':
[('representation', 0.87), ('embeddings', 0.84), ('similar', 0.80), ('meaning', 0.78), ('type', 0.76)]

```

Question 10: Imagine you are a data scientist at a fintech startup. You've been tasked with analyzing customer feedback. Outline the steps you would take to clean, process, and extract useful insights using NLP techniques from thousands of customer reviews. (Include your Python code and output in the code box below.)

Answer:

### Steps to Analyze Customer Feedback Using NLP:

1. **Data Collection:** Gather all customer reviews from sources like databases, CSV files, or APIs.
2. **Text Cleaning and Preprocessing:** Remove noise such as HTML tags, punctuation, numbers, and convert text to lowercase.
3. **Tokenization:** Split text into words or sentences for analysis.
4. **Stopword Removal:** Remove common words like “the,” “is,” and “and” that do not contribute meaningful information.
5. **Stemming/Lemmatization:** Reduce words to their root form (e.g., “running” → “run”) for consistency.
6. **Feature Extraction:** Convert text into numerical representations using techniques like Bag of Words, TF-IDF, or Word Embeddings (Word2Vec, GloVe, etc.).
7. **Sentiment Analysis:** Identify positive, negative, or neutral feedback using pre-trained models or custom classifiers.
8. **Topic Modeling:** Discover common themes or topics using techniques like LDA (Latent Dirichlet Allocation).
9. **Visualization and Insights:** Generate word clouds, frequency distributions, or charts to present key insights to stakeholders.

Code:

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import LatentDirichletAllocation
```

```
# Download NLTK resources
```

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
# Sample dataset (replace with actual customer reviews)
```

```
reviews = [
    "The app is easy to use and very fast!",
    "Customer support was unhelpful and slow.",
    "I love the new features and improvements in the latest update.",
    "The app crashes frequently on my device.",
    "Great experience, but payment processing is confusing."
]
```



```

# 1. Text Cleaning
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^\w\s]', '', text)
    return text

cleaned_reviews = [clean_text(r) for r in reviews]

# 2. Tokenization & Stopword Removal & Lemmatization
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in stop_words]
    return ' '.join(tokens)

processed_reviews = [preprocess(r) for r in cleaned_reviews]

# 3. Feature Extraction using TF-IDF
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(processed_reviews)

# 4. Topic Modeling using LDA
lda = LatentDirichletAllocation(n_components=2, random_state=42)
lda.fit(X)

# Display top words for each topic
def display_topics(model, feature_names, n_top_words):
    for idx, topic in enumerate(model.components_):
        print(f"Topic {idx+1}:")
        print([feature_names[i] for i in topic.argsort()[:-n_top_words - 1:-1]])

print("Top words per topic:")
display_topics(lda, vectorizer.get_feature_names_out(), 5)

O/P:
Top words per topic:
Topic 1:
['app', 'fast', 'easy', 'love', 'feature']
Topic 2:
['support', 'slow', 'crashes', 'payment', 'confusing']

```